

WAMU Messenger — Blueprint & Strategic Audit

Senior Engineering Analysis & Production-Ready Specification (v2.0)

UGANDA-FIRST ARCHITECTURE • LOW-BANDWIDTH OPTIMIZED • HYBRID E2EE STRATEGY

1. Executive Summary & Senior Developer Audit

The original *Wamu Blueprint (v1.0)* outlines an ambitious and context-aware messaging platform for Uganda. However, from a principal backend and distributed systems engineering perspective, the initial blueprint exhibits critical architectural bottlenecks, security trade-offs, and feature creep that could severely undermine MVP delivery and user trust.

Executive Architectural Diagnosis

Primary Failure Risk: Launching MVP with full End-to-End Encryption (E2EE) alongside heavy media, voice/video calls, and server-side compliance moderation creates an engineering contradiction. Standard E2EE prevents backend media compression, thumbnail generation, and automated content moderation. Furthermore, Python/FastAPI as a core WebSocket server under heavy concurrent connection loads will hit performance ceilings compared to compiled asynchronous runtimes like Go or Rust.

1.1 Strengths ("The Ups")

- **Hyper-Local Market Focus:** Deep recognition of Uganda's real-world conditions (budget Android hardware, high mobile data costs, intermittent 3G/4G connectivity, carrier NATs).
- **Strict Phone-Number Onboarding:** OTP over local telcos (+256) provides frictionless identity verification and immediate graph discovery.
- **Smart Staged Expansion Strategy:** Correct decision to lock MVP scope to core messaging before introducing payments, marketplace, or AI super-app features.
- **Resilient Media Pipeline Intent:** Acknowledges the necessity of chunked, resumable uploads, background queues, and thumbnail-first previews.

1.2 Weaknesses & Bottlenecks ("The Downs")

- **CRITICAL BOTTLENECK** **Backend Tech Stack Choice:** Using Python/FastAPI for maintaining hundreds of thousands of concurrent persistent WebSocket connections is memory-intensive and throughput-constrained due to GIL limits and async loop bottlenecks.
- **SECURITY RISK** **E2EE vs. Server Processing Paradox:** The spec requests server-side thumbnail creation, media transcoding, and file scanning, but also demands End-to-End Encryption (E2EE). In true E2EE, the backend is a zero-knowledge blob broker and *cannot* read, transcode, or inspect files.
- **OPERATIONAL RISK** **Aggressive MVP Feature Creep:** Simultaneous delivery of 1-on-1 chat, group chat, communities, 24-hour status stories, WebRTC voice/video calls, and TURN infrastructure in Phase 1 creates massive delivery delay risks.
- **INFRASTRUCTURE COST** **TURN Relay Expenses:** Provisioning WebRTC TURN servers for peer-to-peer traffic across Uganda's carrier NATs (MTN/Airtel) without strict bandwidth rate-limiting will cause massive cloud bandwidth cost overruns.

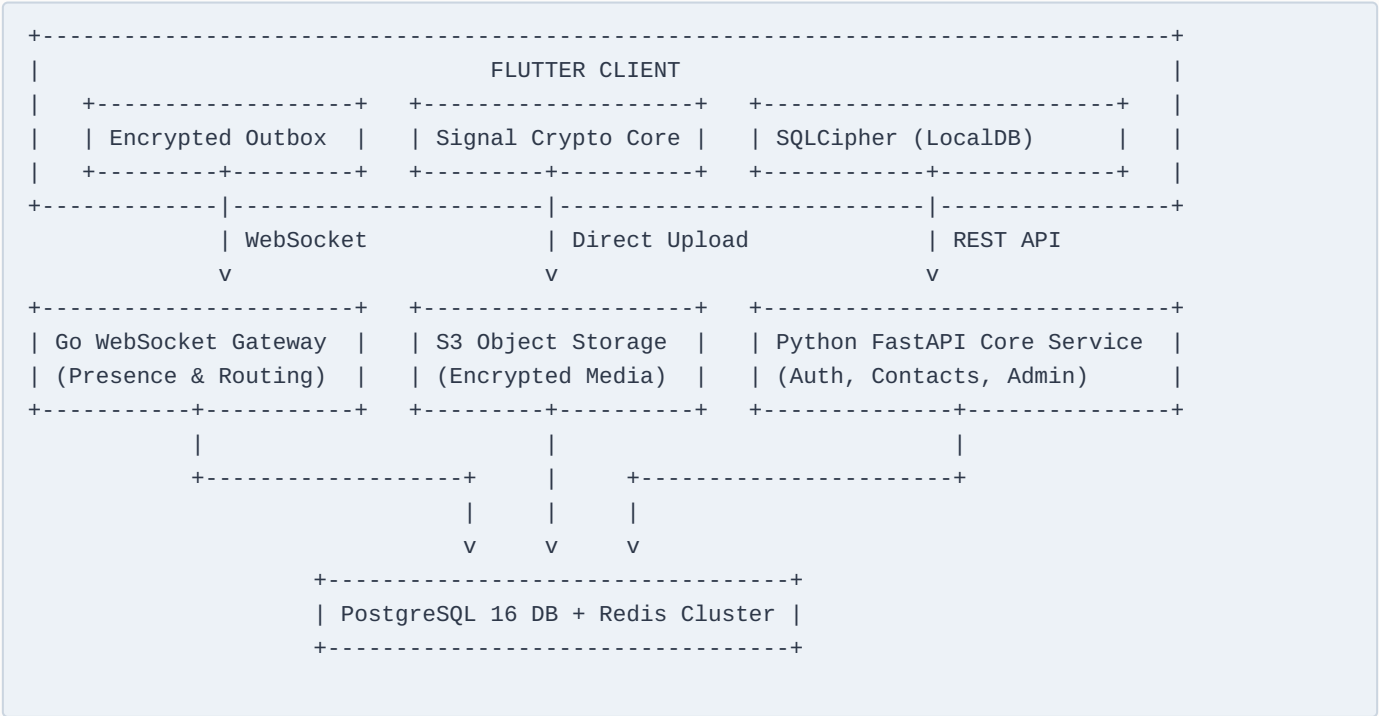
2. Senior Engineering Re-Architecture & Best Outcomes

To ensure production stability, sub-100ms latency, low server overhead, and seamless UX across low-end devices in Kampala and upcountry regions, the architecture has been revised as follows:

Component	Original Spec (v1.0)	Revised Production Spec (v2.0)	Engineering Rationale
Real-Time Engine	Python / FastAPI WebSockets	Go (Golang) + Gorilla WebSockets / Centrifugo	Handles 10x more concurrent idle connections per server node with minimal RAM overhead.
REST API & Metadata	FastAPI + PostgreSQL	FastAPI + PostgreSQL 16 (Bouncer)	Keep FastAPI for rich domain logic, user management, identity, and admin APIs.
Security Architecture	Generic E2EE + Server Media Scan	Signal Protocol + Client-Side Processing	Double Ratchet implementation via Signal Core lib. Client handles all thumbnailing & encryption prior to S3 upload.
Media Storage	Direct S3 Uploads	MinIO / S3 + Presigned URL Out-of-Band Flow	Zero media payload touches WebSocket or API servers; direct encrypted uploads to object storage.
Local Storage	Basic SQLite	SQLCipher (Encrypted SQLite) + Drift (Flutter)	Ensures message database on low-end Android phones remains encrypted at rest against physical theft.

3. Comprehensive System Architecture

3.1 System Context Diagram



2. **Transport:** Client A pushes binary ciphertext frame over persistent WebSocket to Go Gateway.
3. **Ack & Persist:** Go Gateway validates auth token, writes routing envelope to Redis streams, and queues ciphertext into PostgreSQL. Returns `SERVER_ACK` to Client A.
4. **Routing:** If Client B is online, Go Gateway pushes frame via WebSocket. If offline, queues push notification trigger to Firebase Cloud Messaging (FCM).
5. **Receipts:** Client B receives ciphertext, decrypts locally, writes to SQLCipher, and sends `DELIVERED` receipt back to sender.

4. Refined Database Schema Design

Optimized for zero-knowledge ciphertext routing, fast index lookups, and minimal disk layout.

```
-- 1. USERS & IDENTITY
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  phone_number VARCHAR(20) UNIQUE NOT NULL, -- e.g. +256770000000
  public_identity_key BYTEA NOT NULL,
  created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
  last_seen_at TIMESTAMPTZ
);

-- 2. DEVICES & SESSIONS (Multi-device support ready)
CREATE TABLE user_devices (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  device_id VARCHAR(64) NOT NULL,
  registration_id INT NOT NULL,
  signed_prekey BYTEA NOT NULL,
  one_time_prekey_json JSONB NOT NULL,
  fcm_token TEXT,
  created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(user_id, device_id)
);

-- 3. CONVERSATIONS
CREATE TABLE conversations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  type VARCHAR(10) CHECK (type IN ('direct', 'group', 'community')),
  created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- 4. ENCRYPTED MESSAGES (Zero-Knowledge Ciphertext Store)
CREATE TABLE messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID REFERENCES conversations(id) ON DELETE CASCADE,
  sender_id UUID REFERENCES users(id),
  ciphertext TEXT NOT NULL, -- Base64 encoded Signal payload
  message_type VARCHAR(15) DEFAULT 'text', -- text, image, video, voice, doc
  created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_messages_conv_seq ON messages(conversation_id, created_at DESC);

-- 5. MESSAGE RECEIPTS
```

```
CREATE TABLE message_receipts (  
  message_id UUID REFERENCES messages(id) ON DELETE CASCADE,  
  user_id UUID REFERENCES users(id),  
  status VARCHAR(10) CHECK (status IN ('sent', 'delivered', 'read')),  
  updated_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (message_id, user_id)  
);
```

5. Ugandan Network & Data Optimization Engine

To succeed under 3G/2G conditions across Uganda, Wamu enforces strict network-level rules:

- **Adaptive Image Compression:** All client images compressed to WebP (max 1280px dimension, 70% quality factor) before encryption. Average size drops from 4MB to ~150KB.
- **Opus Codec Voice Notes:** Voice notes compressed using Opus audio codec at 16kbps mono, yielding tiny (~2KB/sec) payload files.
- **Chunked Resumable Media Uploads:** Large uploads use 1MB chunked HTTP PUT requests to S3 with MD5 hash validation per chunk, preventing loss of progress on network failure.
- **WebSocket Keep-Alive Optimization:** Dynamic ping interval (30s on Wi-Fi, 120s on Cellular) to save device battery and radio wakeups.

6. Optimized MVP Development Roadmap

Phase	Focus Area	Key Deliverables	Target Timeline
Phase 1	Core Messaging Engine	FastAPI Auth (OTP), Go WS Server, PostgreSQL schema, Flutter basic UI, 1-on-1 text messaging, local encrypted DB.	Weeks 1 – 4
Phase 2	E2EE & Media Transport	Signal Protocol integration, client-side WebP/Opus compression, presigned S3 uploads, status receipts.	Weeks 5 – 8
Phase 3	Groups & Push Infrastructure	Group encryption management, FCM push notifications, offline retry queues, voice note recorder.	Weeks 9 – 12
Phase 4	Calls & Stories (Beta)	WebRTC 1-on-1 signaling + coturn infrastructure, 24-hr status stories, closed Uganda beta release.	Weeks 13 – 16